

Semana 5
ISIS-1611: Inteligencia Artificial

Carla González

2 de julio de 2026

1. Problemas de Satisfacción de restricciones

1.1. Diagrama de Arbol

En la clase anterior se definió un CSP como una tripla (*variables, dominios, restricciones*) y se vió la **busqueda backtracking**, que es como un DFS + ordenamiento de variables + falla inmediata ante violación; junto con dos mejoras clave:

- **Filtrado** mediante *consistenciadearcos*; revisar si para cada valor de X, existe algún valor compatible en Y que detecta fallas inevitables antes que la simple revisión forward.
- **Ordenamiento** MRV (*Minimun Remaining Values*) para variables y LCV (*Least constraining value*) para valores

Ejemplo 1.1. Con dominios iniciales rojo,verde,azul y restricciones de desigualdad entre regiones vecinas, al forzar consistencia de arcos sobre el grafo WA NT SA Q NSW V se observa el efecto en cadena: Si el dominio de X pierde un valor, todos los vecinos de X se deben volver a revisar,

Arco revisado	Resultado
V → NSW	Consistente
SA → NSW	Consistente
NSW → SA	Inconsistente, se elimina azul del dominio de NSW
V → NSW	Como NSW cambi+o, se vuelve a ervisar se elimina rojo de V

Cuadro 1: Efecto en cadena al forzar consistencia de arcos.

porque una restriccion que antes era satisfacible puede dejar de serlo

Forzar consistencia de arcos sobre todo el CSP **no garantiza** encontrar la solución. al terminar el proceso puede que a quede exactamente una solución, b queden multiples soluciones sin decidir entre ellas o c el problema no tenga soluci'ón y el algoritmo no lo sepa. Por eso la consistencia se arcos se corre dentro de una busqueda backtracking no en lugar de ella.

1.2. k-consistencia

La consistencia de arcos es apenas un caso particular de na idea más general, ir aumentando el grado de consistencia que se exige, de nodos a arcos, de arcos a trios de variables y así sucesivamente.

Definición 1.1. Un CSP es k-consistente si, para cualquier conjunto de k-1 variables y cualquier asignación consistente a esas k-1 variables, siempre existe un valor consistente para extender la asignación a una k-esima variable cualquiera.

Nivel	Que garantiza
1-consistencia (consistencia de nodos)	Cada dominio ya respeta las restricciones unarias del nodo
2-consistencia (consistencia de arcos)	Toda asignación consistente a un nodo se extiende al otro
3-consistencia (consistencia de caminos)	Todo par de nodos consistentes se pued extender a un tercero
k-consistencia	Se cumple simultaneamente para 1,2,...,k

A mayor k, el filtrado poda mas el arbol de bsuqeuda, pero tambien es **mas caro de computar**. No hay almuerzo gratis, es un tradeoff entre tiempo invertido en propagación de restricciones y tiempo inertido en backtracking.

1.3. Forward Checking

Es una alternativa más barata que forzar consistencia de arcos completa. Cada vez que se asigna un valor a X_i , se recorren las variables **vecinas no asignadas** de X_i y se elimina de sus dominios cualquier valor inconsistente con la asignación recién hecha.

Definición 1.2 (Forward Checking). Al asignar $X_i \leftarrow v$, para cada variable X_j vecina de X_i y aún sin asignar, eliminar de D_j todo valor que no sea consistente con $X_i = v$. Si algún D_j queda vacío, se detecta el fallo inmediatamente sin necesidad de llegar a esa variable.

La diferencia con la consistencia de arcos (AC-3) es que forward checking **solo revisa los arcos que salen de la variable recién asignada**, una sola vez. No propaga el efecto en cadena hacia el resto del grafo como sí lo hace AC-3 (que vuelve a revisar los vecinos de un vecino cuando su dominio cambia). Por eso forward checking es más barato pero detecta menos inconsistencias por adelantado; es un punto intermedio entre backtracking simple (0 chequeo hacia adelante) y AC-3 completo (propagación total).

1.4. Estructura del grafo de restricciones

Cuando el grafo de restricciones de un CSP es un **árbol** (sin ciclos), el problema se puede resolver en tiempo $\mathcal{O}(nd^2)$, sin ningún backtracking.

Definición 1.3 (CSP con estructura de árbol). 1. Elegir una variable como raíz y ordenar el resto en un orden topológico $X_1, \dots, X_n$ tal que cada variable aparezca después de su padre en el árbol.

2. Recorrer las variables **de atrás hacia adelante** (j de n a 2) y forzar consistencia de arco entre X_j y su padre $\text{Padre}(X_j)$, eliminando del dominio del padre los valores que ya no tengan soporte.

3. Recorrer las variables **de adelante hacia atrás** (j de 1 a n) asignando a cada X_j cualquier valor de su dominio consistente con el valor ya asignado a $\text{Padre}(X_j)$.

Como el grafo quedó dirigido y arco-consistente antes de asignar, el segundo recorrido nunca falla: nunca hay que retroceder.

La mayoría de los CSP reales no son árboles, pero se pueden **acercar** a esa estructura:

- **Cutset conditioning**: encontrar un conjunto pequeño de variables (*cutset*) tal que, al eliminarlas del grafo, lo que queda es un árbol. Se prueban todas las asignaciones consistentes posibles para el cutset y, para cada una, se resuelve el árbol restante en $\mathcal{O}(nd^2)$. Si el cutset tiene tamaño c , el costo total es $\mathcal{O}(d^c \cdot (n - c)d^2)$: exponencial en c , pero manejable si c es pequeño respecto a n .
- **Tree decomposition**: agrupar variables en subproblemas conectados en forma de árbol (cada subproblema se resuelve por separado) para luego combinar soluciones consistentes entre subproblemas vecinos.

1.5. Búsqueda local para CSP: Min-conflicts

A diferencia de backtracking (que construye la asignación variable por variable), min-conflicts parte de una **asignación completa** (posiblemente inconsistente) y la va reparando.

Definición 1.4 (Min-conflicts). 1. Generar una asignación inicial completa, asignando a cada variable un valor cualquiera de su dominio (puede violar restricciones).

2. Repetir hasta encontrar una solución o alcanzar un número máximo de pasos:

- a) Elegir al azar una variable X_i que esté en conflicto (viola alguna restricción).
- b) Reasignarle el valor de su dominio que **minimice el número de conflictos** con las demás variables (rompiendo empates al azar).

Ejemplo 1.2 (Min-conflicts en n-reinas). Con las 8 reinas ya puestas una por columna (asignación inicial con conflictos), en cada paso se elige una reina que ataque a otra y se mueve dentro de su columna a la fila con menos reinas atacándola. En la práctica, resuelve instancias de miles de reinas en muy pocos pasos.

Es muy eficiente en la práctica incluso para CSPs grandes, pero **no garantiza completitud**: puede quedar atascado y nunca encontrar una solución aunque exista (no hay noción de retroceso ni de dominios que se agotan).

2. Ejercicio: Coloreado del mapa de Australia

Ejercicio 2.1. Aplicar backtracking con MRV (y desempate por grado) y propagación tipo forward checking al problema de colorear el mapa de Australia con 3 colores {rojo, verde, azul}, exigiendo que regiones vecinas tengan colores distintos. Regiones: WA, NT, SA, Q, NSW, V, T (Tasmania es una isla, no tiene vecinos). Adyacencias: WA-NT, WA-SA, NT-SA, NT-Q, SA-Q, SA-NSW, SA-V, Q-NSW, NSW-V.

Solución:

Todas las variables (salvo T) parten con $|D| = 3$: hay empate en MRV, así que se desempata con la **heurística de grado** (más restricciones sobre variables sin asignar todavía). SA tiene grado 5, el más alto de todo el grafo, así que se asigna primero.

Paso	Asignación	Efecto de la propagación
1	SA = rojo	Se elimina rojo de WA, NT, Q, NSW, V
2	Q = verde	Se elimina verde de NT y de NSW
3	NT = azul	Se elimina azul de WA
4	WA = verde	Sin vecinos nuevos por revisar
5	NSW = azul	Se elimina azul de V
6	V = verde	Sin vecinos nuevos por revisar
7	T = rojo	T no tiene restricciones, cualquier color sirve

Cuadro 2: Traza de backtracking + MRV + forward checking sobre el mapa de Australia.

En cada paso, después de asignar, la variable que queda con el dominio más pequeño (a veces de tamaño 1) es la siguiente elegida por MRV, así que no hace falta romper más empates. La propagación evita que en este caso se necesite retroceder: ningún dominio queda vacío en el camino.

Solución final: SA = rojo, Q = verde, NT = azul, WA = verde, NSW = azul, V = verde, T = rojo (o cualquier color, T no tiene vecinos).

3. Agentes Lógicos: Introducción

3.1. Agentes basados en conocimiento

Un agente basado en conocimiento mantiene una **base de conocimiento** (KB): un conjunto de oraciones en algún lenguaje de representación que expresan hechos sobre el mundo. El agente decide qué acción tomar consultando (ASK) e incorporando percepciones nuevas (TELL) a la KB, usando un **motor de inferencia**.

Definición 3.1 (Agente basado en conocimiento). En cada ciclo el agente: (1) le dice (TELL) a la KB lo que percibió, (2) le pregunta (ASK) cuál es la mejor acción dado lo que sabe, y (3) le dice a la KB qué acción ejecutó. Todo el conocimiento sobre el dominio vive en la KB, no en el código del agente, así que se puede razonar y no solo reaccionar.

$KB \vdash \alpha$ (la KB **deriva** α sintácticamente, con un procedimiento de inferencia)

Un procedimiento de inferencia es **sólido** (*sound*) si solo deriva oraciones que realmente son consecuencia lógica de la KB, y es **completo** si es capaz de derivar toda oración que sea consecuencia lógica.

3.2. Lógica proposicional

Sintaxis: oraciones atómicas (símbolos proposicionales, cada uno verdadero o falso) combinadas con conectivos lógicos $\neg$ (negación), $\wedge$ (conjunción), $\vee$ (disyunción), $\Rightarrow$ (implicación), $\Leftrightarrow$ (bicondicional), para formar oraciones complejas.

Semántica: un **modelo** es una asignación de verdadero/falso a cada símbolo proposicional. Dado un modelo, el valor de verdad de cualquier oración se calcula recursivamente con las tablas de verdad de los conectivos.

- Una oración es **válida** (tautología) si es verdadera en **todos** los modelos (ej. $P \vee \neg P$).
- Una oración es **satisfacible** si es verdadera en **al menos un** modelo.
- Una oración es **insatisfacible** si no es verdadera en ningún modelo (ej. $P \wedge \neg P$).
- **Consecuencia lógica (entailment):** $KB \models \alpha$ si en todo modelo donde KB es verdadera, α también lo es. Equivalentemente, $KB \models \alpha$ si y solo si $KB \wedge \neg \alpha$ es insatisfacible.

Ejemplo 3.1 (Model checking). Para decidir si $KB \models \alpha$ por enumeración de modelos: se listan todas las combinaciones de verdad de los símbolos que aparecen en KB y en α (tabla de verdad completa), y se revisa que en toda fila donde KB sea verdadera, α también lo sea. Es correcto pero exponencial en el número de símbolos.

3.3. Formas normales: CNF y DNF

- **CNF** (Forma Normal Conjuntiva): conjunción de cláusulas, donde cada cláusula es una disyunción de literales. Es la forma que usan la mayoría de los algoritmos de inferencia (resolución, DPLL, etc.).
- **DNF** (Forma Normal Disyuntiva): disyunción de términos, donde cada término es una conjunción de literales.

Pasos para convertir cualquier oración a CNF:

1. Eliminar $\Leftrightarrow$: reemplazar $\alpha \Leftrightarrow \beta$ por $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.
2. Eliminar $\Rightarrow$: reemplazar $\alpha \Rightarrow \beta$ por $\neg \alpha \vee \beta$.
3. Meter las negaciones hacia adentro usando De Morgan ($\neg(\alpha \wedge \beta) \equiv \neg \alpha \vee \neg \beta$, $\neg(\alpha \vee \beta) \equiv \neg \alpha \wedge \neg \beta$) y eliminar dobles negaciones.
4. Distribuir $\vee$ sobre $\wedge$ hasta que quede una conjunción de disyunciones de literales.

3.4. El mundo del Wumpus

Ambiente clásico para introducir agentes lógicos: una cueva en cuadrícula (típicamente 4×4), con un Wumpus que mata al agente si entra en su casilla, pozos que lo hacen caer, y oro que debe recoger y sacar. El agente empieza en $(1, 1)$ y solo percibe su casilla actual.

- **Percepciones:** *stench* (hedor, si el Wumpus está en una casilla adyacente), *breeze* (brisa, si hay un pozo adyacente), *glitter* (brillo, si hay oro en la casilla), *bump* (chocó con pared), *scream* (grito, al matar al Wumpus con una flecha).
- **Acciones:** avanzar, girar izquierda/derecha, agarrar, disparar, subir (para salir de la cueva).

Ejemplo 3.2 (Inferencia en el mundo del Wumpus). Si en $(1, 1)$ el agente no percibe ni hedor ni brisa, la regla $Breeze_{(x,y)} \Leftrightarrow \bigvee \text{Pit}_{\text{vecino}}$ (y la equivalente para *Stench* y el Wumpus) permite concluir que **ninguna** casilla vecina, $(1, 2)$ y $(2, 1)$, tiene pozo ni Wumpus: son seguras para avanzar. Si luego en $(2, 1)$ el agente sí percibe brisa (pero no hedor), infiere que hay un pozo en $(3, 1)$ o en $(2, 2)$, pero **no sabe en cuál** de los dos sin más información; ese es justamente el tipo de razonamiento bajo información parcial que la KB tiene que resolver combinando todas las percepciones acumuladas, no solo la última.